

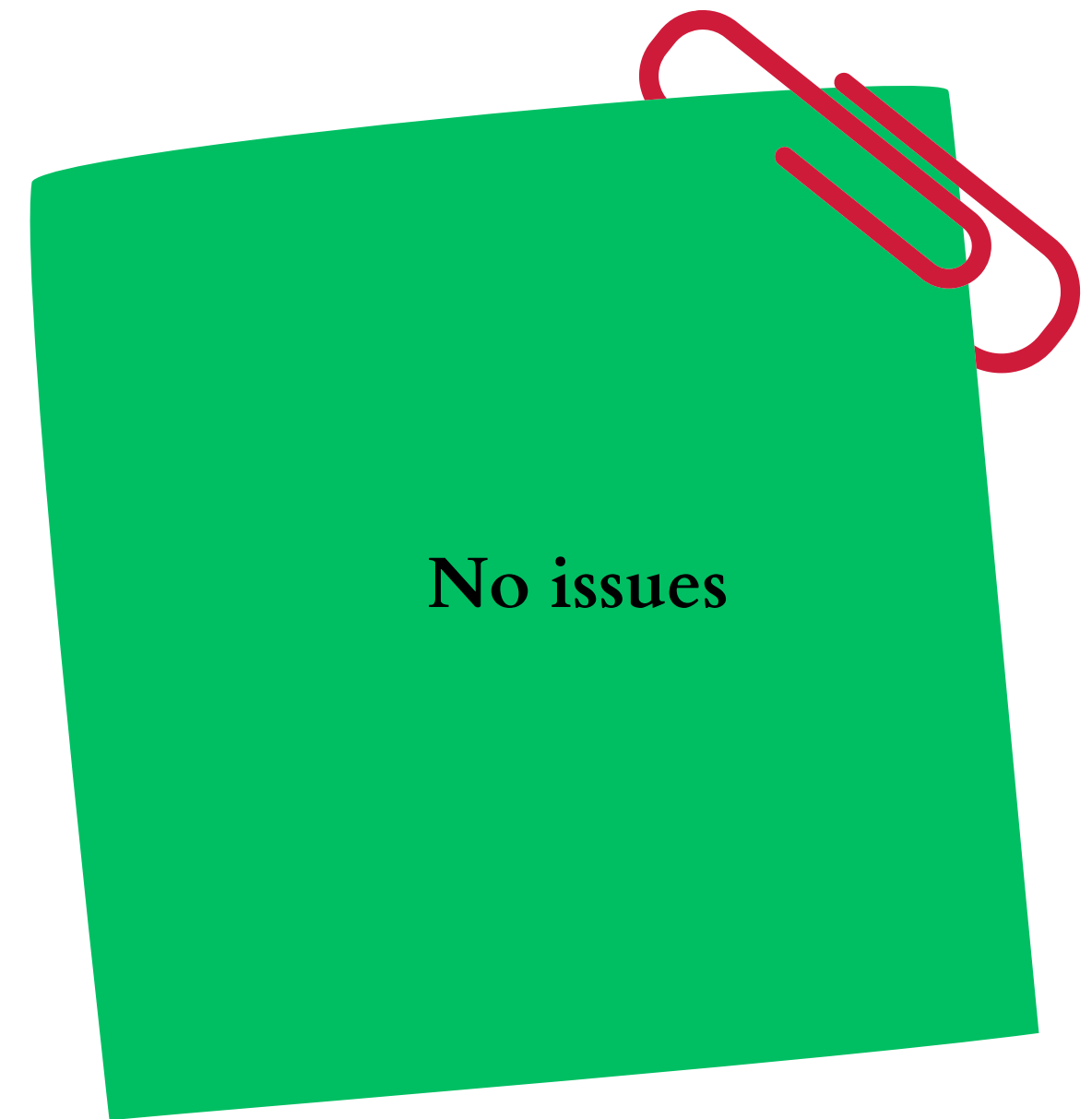
Welcome

Day 1

Dr. Rachel DuBose

University Libraries Research Computing Services

Logistics



Next break will be at noon for Lunch²!

Goals



YOUR FEEDBACK IS
IMPORTANT

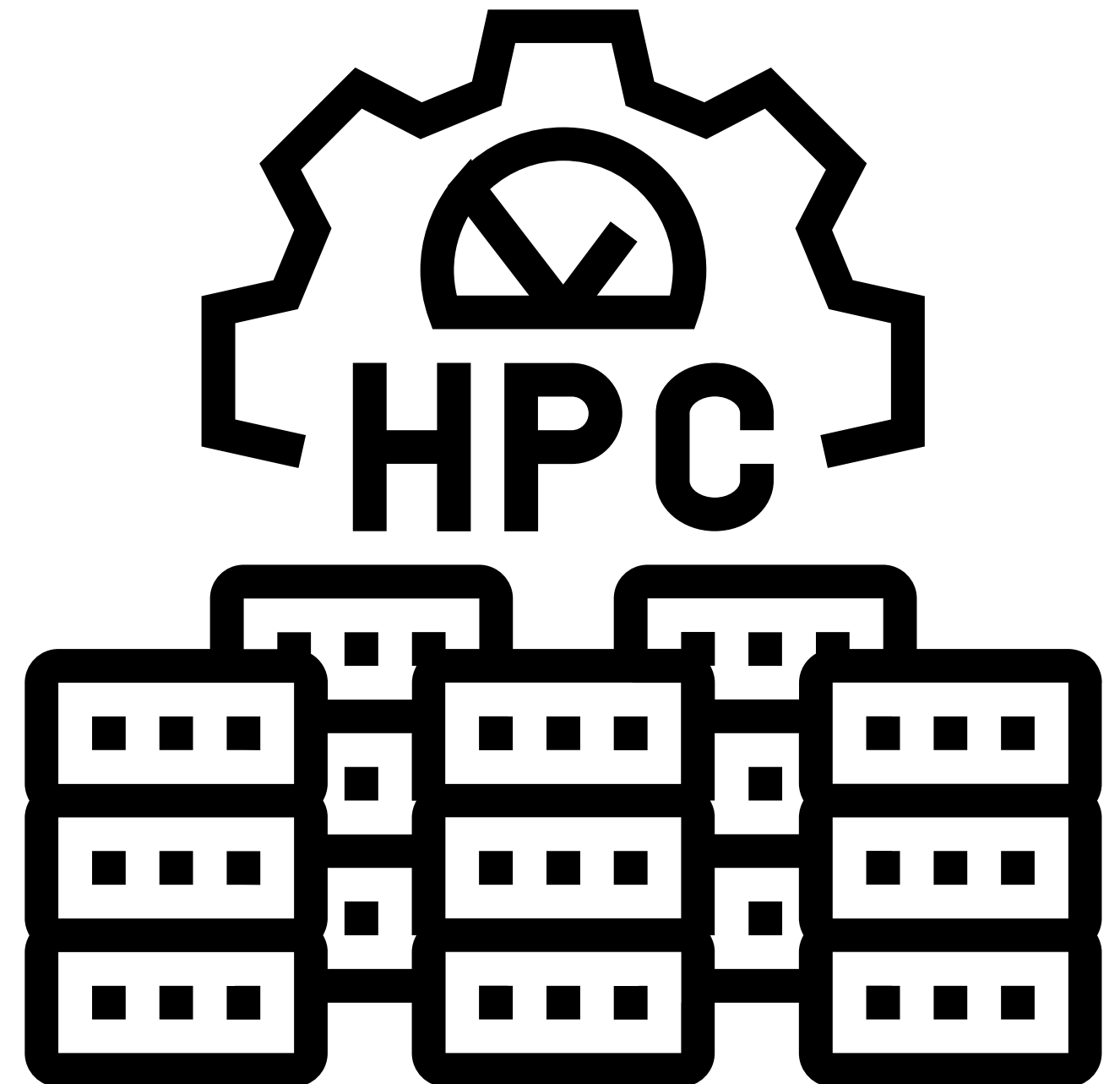


What questions do you have?

Overview

- Conceptual overview of resources
- Help & Policies
- Connecting to UAHPC
- Running your first job on UAHPC
- SFTP
- Modules and package managers

UAHPC is a set of high-performance servers...
...connected by a very high-speed network
...with storage attached to the network
...and resources that can be scheduled.



When to utilize HPC

HPC focuses on any computing that challenges the resources at hand

When your tasks are taking too long

When large tasks need to be divided into smaller tasks and executed simultaneously
(Parallel Computing)

When a task requires rapid data transfer between computing nodes to reduce bottlenecks

How to get help

1. Email ITSD: itsd@ua.edu

2. HPC Portal (may be faster)

3. <https://guides.lib.ua.edu/RCS/hpc>

4. Book an appt with me:

<https://ua.libcal.com/appointments/dubose>

Research Computing Services and Resources

Overview of UA Libraries Research Computing Services and Resources

Overview

Workshops

General Programming

Development Tools

High Performance Computing

Linux

Generative Artificial Intelligence

Machine Learning

Automation

Contacts

Search this Guide

Search

Overview

High performance computing involves the use of supercomputers and computing clusters to solve advanced computational problems. High performance computing takes advantage of parallel computing to run multiple tasks simultaneously on numerous computer servers or processors, enabling high speed and high throughput computational solutions.

See the UA High Performance Computing and Data Center: <https://hpc.ua.edu/>

E-Books



High-Performance Big Data Computing by Dhabaleswar K. Panda; Xiaoyi Lu; Dipti Shankar
ISBN: 9780262046855
Publication Date: 2022-08-02



Parallel Programming for Modern High Performance Computing Systems by Pawel Czarnul
ISBN: 9781138305953
Publication Date: 2018-02-28



High-Performance Computing by Laurence T. Yang; Minyi Guo
Call Number: QA76.88 .H538 2006
ISBN: 0471732702
Publication Date: 2006-01-24



High Performance Computing by Thomas Sterling; Maciej Brodowicz; Matthew Anderson
ISBN: 0124202152
Publication Date: 2017-12-05

Tutorials and Guides

[HPC Carpentry Lessons](#)
Repository of peer-reviewed, short-format, lessons that use the teaching approach and lesson design from The Carpentries.

[National Computational Infrastructure HPC Tutorial](#)
Comprehensive set of lessons providing introduction to High-Performance Computing (HPC) concepts, tools, and practices.

[LLNL HPC Tutorials](#)
Tutorials developed and supported by Lawrence Livermore National Lab.

Key policies to know

- UAHPC is accessible from the University of Alabama IP address space.
- From off-campus, you must use the VPN to get to the UA network. The system provides services via SSH/SFTP.
- Responsible use is required.

[Acceptable Use Policy | High Performance Computing and Data Center](#)

[Information Protection Procedure | Office of Information Technology](#)

Connecting to a server

To connect to a Linux server, you need to SSH (Secure Shell) into the server.

SSH is a network protocol that allows you to securely connect to a remote server over an unsecured network.

IP name or hostname of server
Username
Password (or SSH key)

To SSH into a server, open a terminal or Powershell prompt and type the following command:

```
ssh username@hostname
```

Connecting to UAHPC

- 1) Open your terminal on Linux and Mac; on Windows, use Powershell
- 2) Enter the following command to ssh into the HPC cluster:

```
ssh myBamaID@uahpc.ua.edu
```

username

hostname

- 3) Enter your myBama password
- 4) Complete Duo two-factor authentication

No issues

Help!

Anatomy of a Linux command



The diagram shows a terminal prompt and a command: `[myBamaID@uahpc-login001 ~]$ ls -l test`. Three labels with arrows point to parts of the command: 'Command' points to 'ls', 'Option' points to '-', and 'Argument' points to 'test'.

Command: Built-in instruction you want to run.

Option: Also called "flags". These modify the behavior of the command and start with either a - or --.

Argument: This is the target that the command acts upon. It can be a filename, path, input, etc.

Basic Commands

- **pwd** - Print working directory or where you are currently located in the file system
- **ls** - List the contents of the current directory
- **cd** - Change directory
- **mkdir** - Create a new directory
- **rm** - Remove a file
- **rmdir** - Remove a directory
- **cp** - Copy a file
- **mv** - Move a file
- **cat** - Display the contents of a file

How to get help

help - Get help on a command

```
[myBamaID@uahpc-login001 ~]$ help -d cd  
cd - Change the shell working directory
```

man - Display the manual page for a command

```
[myBamaID@uahpc-login001 ~]$ man cat
```

```
CAT(1)                                User Commands  
  
NAME  
    cat - concatenate files and print on the standard output  
  
SYNOPSIS  
    cat [OPTION]... [FILE]...  
  
DESCRIPTION  
    Concatenate FILE(s) to standard output.  
  
    With no FILE, or when FILE is -, read standard input.  
  
    -A, --show-all  
        equivalent to -vET  
  
    -b, --number-nonblank  
        number nonempty output lines, overrides -n  
  
    -e      equivalent to -vE  
  
    -E, --show-ends  
        display $ at end of each line  
  
    -n, --number  
        number all output lines  
  
    -s, --squeeze-blank  
        suppress repeated empty output lines
```

info - offers more detailed and structured info than man command

```
[myBamaID@uahpc-login001 ~]$ info --show-options nano
```

```
Next: Command-line Options,  Prev: Introduction,  Up: Top  
  
2 Invoking  
*****  
  
The usual way to invoke 'nano' is:  
  
    nano [FILE]  
  
    But it is also possible to specify one or more options (see the next  
section), and to edit several files in a row.  Additionally, the cursor  
can be put on a specific line of a file by adding the line number with a  
plus sign before the filename, and even in a specific column by adding  
it with a comma.  So a more complete command synopsis is:  
  
    nano [OPTION]... [[+LINE[,COLUMN]|+,COLUMN] FILE]...  
  
    Normally, however, you set your preferred options in a 'nanorc' file  
(*note Nanorc Files:).  And when using 'set positionlog' (making 'nano'  
remember the cursor position when you close a file), you will rarely  
need to specify a line number.  
  
    As a special case: when instead of a filename a dash is given, 'nano'  
will read data from standard input.  This means you can pipe the output  
of a command straight into a buffer, and then edit it.
```

File System Navigation: step 1

The Linux file system is organized in a hierarchical structure with the root directory (/) at the top.

The user will start in their home directory: /home/<myBamaID>

When you login to the server, enter the command pwd to see what directory you're currently in:

```
[myBamaID@uahpc-login001~] $pwd
```

```
/home/myBamaID
```


File System Navigation: step 2

Use the `ls` command to know what files are in the current directory.

```
[myBamaID@uahpc-login001 ~] $ ls
```

```
[myBamaID@uahpc-login001 ~] $
```

Since we have not created any files or directories it won't return anything.

File System Navigation: step 3

Use `mkdir <directory name>` to create a directory.

```
[myBamaID@uahpc-login001 ~]$ mkdir test
```

```
[myBamaID@uahpc-login001 ~]$
```

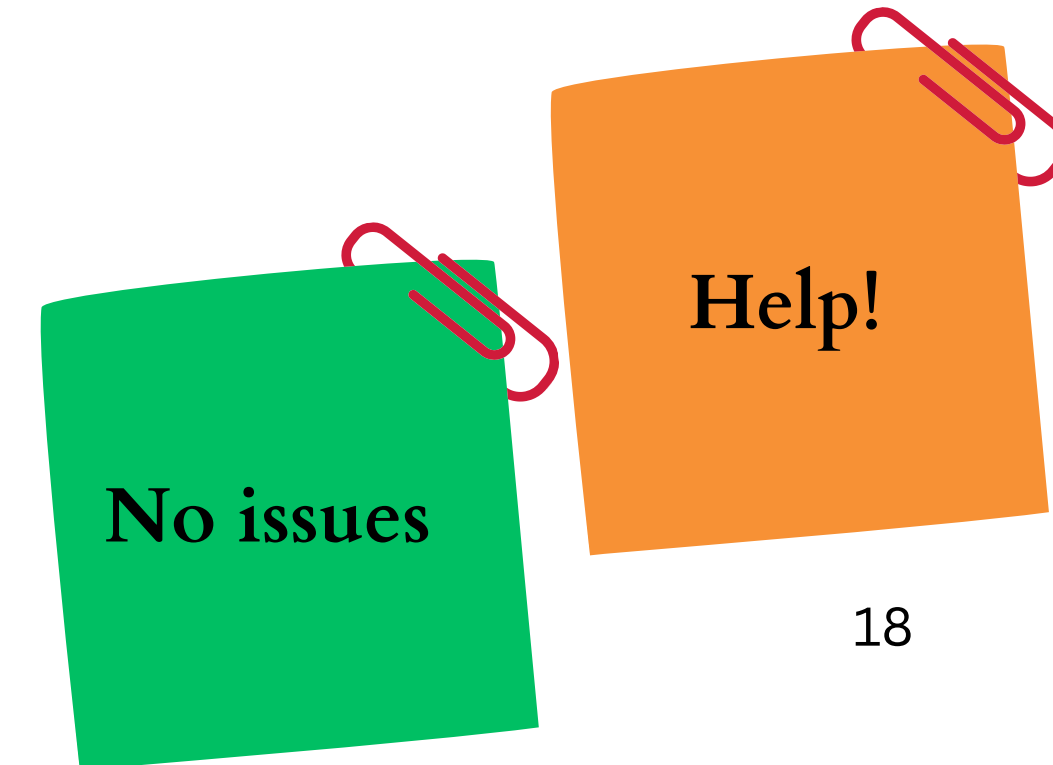
If it was successful, you will see the system prompt like in the example above.

Now, let's do `ls` again:

```
[myBamaID@uahpc-login001 ~]$ ls
```

test

We can now see the directory we just created.



File System Navigation: step 4

Use `cd <directory-name>` to go into the test directory we created and enter `pwd` before and after to see this.

```
[myBamaID@uahpc-login001 ~]$ pwd
```

```
/home/myBamaID
```

```
[myBamaID@uahpc-login001 ~]$ cd test
```

```
[myBamaID@uahpc-login001 test]$ pwd
```

```
/home/myBamaID/test
```

To go back one directory, use `cd ..` where `..` represents the parent directory. Use `pwd` to see the changes take place.

```
[myBamaID@uahpc-login001 test]$ cd ..
```

```
[myBamaID@uahpc-login001 ~]$ pwd
```

```
/home/myBamaID
```

You can also use `cd ~` to go back to your home directory from anywhere in the file system.

Let's go back into the test directory once more.

```
[myBamaID@uahpc-login001 ~]$ cd test
```

Nano: Text editor

To create/edit a file, we can use text editors such as nano, vim, or emacs. We will use nano as it is easy to use.

To edit a file using nano, type the following command: **nano <filename>**

If the file does not currently exist in the directory, nano will create a new one.

When you enter the command, you should see a window like this:

A screenshot of the GNU nano 2.9.8 text editor interface. The window has a title bar at the top with "GNU nano 2.9.8" on the left and "file.txt" on the right. The main editing area is a large black rectangle with a single white cursor character at the top left. At the bottom, there is a status bar with various keyboard shortcuts and their functions. The shortcuts are arranged in two rows. The first row includes: ^G Get Help, ^O Write Out, ^W Where Is, ^K Cut Text, ^J Justify, ^C Cur Pos, M-U Undo, and M-A Mark Text. The second row includes: ^X Exit, ^R Read File, ^\ Replace, ^U Uncut Text, ^T To Spell, ^_ Go To Line, M-E Redo, and M-6 Copy Text. A "[New File]" prompt is visible in the center of the status bar.

Nano: Text editor

The most important commands:

Ctrl + O : Save the file

Ctrl + X : Exit the editor

To learn more about nano, you can type `man nano` in the terminal to view the manual page for the nano command, or you can check out the nano [documentation](#).

Creating a Python file

```
nano hpc_hello.py
```

in nano window:

```
#!/usr/bin/env python3  
# hpc_hello.py: Print out a greeting  
  
print('Hello World, from UAHPC!')
```

Ctrl + O # saves the file (after you hit Ctrl + O, hit enter again)

Ctrl + X # exits nano

To submit a job, we use a scheduler

On an HPC system, the scheduler manages which jobs run where and when.

Three **key functions** of a scheduler:

1. Allocates access to resources for users for some duration of time to perform work.
2. Provides a framework to start, execute, and monitor work.
3. Arbitrates contention for resources by managing queue of work.

Slurm

Any command or set of commands we want to run is a **job**.

The process of using a scheduler to run the job is called **batch job submission**.

We need to monitor our jobs and use of resources.

Job submission commands

sbatch – submit a batch script for later execution

salloc – obtain a job allocation

srun – obtain a job allocation and execute an application*

Accounting commands

sacct – display accounting data

sreport – report job usage and cluster utilization

Job management commands

squeue – view info about jobs

sinfo – view all info about nodes and partitions

scancel – signal jobs

To create a batch script for batch job submission

```
nano hello.batch
```

In nano window:

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --mem=1G
#SBATCH --job-name=test_hpc_job
#SBATCH --partition main
#SBATCH --qos main
#SBATCH --time=1:00
#SBATCH --error errors.%A
#SBATCH --output output.%A
```

```
python ./hpc_hello.py
```

Ctrl + O # saves the file (after you hit Ctrl + O, hit enter again)

Ctrl + X # exits nano

Once you have exited nano,
run the Python file with:
sbatch hello.batch

You can check on the job with:
squeue -u myBamaID

View the results of your first job on UAHPC!

To view a file, you can use: **cat <filename>**

Use this to view the output you just created:

```
[myBamaID@uahpc-login001 test]$ ls
```

```
[myBamaID@uahpc-login001 test]$ cat output.%A
```

%A denotes job number



No issues



Help!

SFTP: SSH File Transfer Protocol

A network protocol used for secure file transfer over a connection

Ensures data is transmitted encrypted

Differs from SCP (Secure Copy Protocol)

- Offers a range of commands to manage files and directories
- Establishes a persistent connection during file transfer, allowing resumption of interrupted transfers and multiple file management

Basic SFTP Commands

ls - Lists the contents of the current directory on the remote server. You can also **ls** to specify a path:

ls /path/to/remote/directory

lls - Lists the contents of the current directory on your local machine. You can also use **lls** to specify a path:

lls /path/to/local/directory

cd - Changes the current directory on the remote server

cd /path/to/remote/directory

lcd - Changes the current directory on your local machine

lcd /path/to/local/directory

pwd - Prints the current working directory on the remote server

pwd

lpwd - Prints the current working directory on your local machine

lpwd

mkdir - Creates a new directory on the remote server

mkdir new_remote_directory

lmkdir - Creates a new directory on your local machine

lmkdir new_local_directory

exit - To exit an SFTP session

SFTP: SSH File Transfer Protocol

In a new terminal, to connect to the SFTP server:

```
sftp myBamaID@uahpc.ua.edu
```

You will be prompted to enter your password and complete Duo two factor verification.

After successfully connecting to the server, you should see a system prompt like this: `sftp>`

SFTP: Copying files to and from a server

Local machine to remote server: put

```
put /path/to/localfile path/to/destination/folder
```

Remote server to local machine: get

```
get /path/to/remotefile path/to/local/directory
```

To copy a folder, use `-r` flag

SFTP Exercise

1. Navigate to Box folder: <https://alabama.app.box.com/folder/383567545827>
2. Download primes.batch and primesspiral.py
3. In new terminal, type the following and complete authentication steps:

```
sftp myBamaID@uahpc.ua.edu
```

4. SFTP the files, using these commands one by one:

```
put C:\Users\myBamaID\Downloads\spiral.batch /home/myBamaID/test
```

```
put C:\Users\myBamaID\Downloads\primesspiral.py /home/myBamaID/test
```

Don't close that terminal window!



No issues



Help!

Modules on HPC are a way to manage different software environments

Commands

module avail – lists all available modules

module load – load a module

module list – lists currently loaded modules

module purge – unloads all modules

Reasons we use modules

- allow users to load and unload software packages as needed
- helps in managing different versions of software and their dependencies



Using modules

BACK IN SSH TERMINAL

To load a module, use:

```
[myBamaID@uahpc-login001]$ module load python
```

```
[myBamaID@uahpc-login001]$ module list
```

```
Currently Loaded Modulefiles:
```

```
1) slurm/uahpc/24.05.4  2) gcc/11.2.0  3) python/python3/3.11.8.test123
```

```
[myBamaID@uahpc-login001]$ module unload python
```

```
[myBamaID@uahpc-login001]$ cat primesspiral.py
```

```
import matplotlib.pyplot as plt
import numpy as np
```

← Required packages are at top

```
# Check if a number is prime
```

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True
```

```
# Generate spiral coordinates
```

```
def generate_spiral(n_points, spacing=0.5):
    theta = np.linspace(0, n_points * spacing, n_points)
    r = theta
    x = r * np.cos(theta)
    y = r * np.sin(theta)
    return x, y
```

← Where numpy is used

```
# Number of points to plot
```

```
N = 1000
```

```
# Generate spiral
```

```
x, y = generate_spiral(N)
```

```
# Plot
```

```
plt.figure(figsize=(8, 8))
```

```
for i in range(N):
```

```
    if is_prime(i):
```

```
        plt.plot(x[i], y[i], 'ro', markersize=3) # red dot for prime
```

```
    else:
```

```
        plt.plot(x[i], y[i], 'k.', markersize=1) # small black dot for non-prime
```

← Where matplotlib is used

How do we get the packages we need?

We use a package manager.

A package manager is a system that enables you to set up a **new, project specific software environment** containing specific software versions as well as the versions of **additional packages and required dependencies** that are **all mutually compatible**.

Why use a package manager?

- They help resolve dependency issues by allowing you to use different versions of a package for different projects.
- Your projects are now self-contained and reproducible by capturing all package dependencies in a single requirements file.
- Allows you to install packages on a host on which you do not have admin privilege.

Setting up our Environment



Run this code line by line:

```
cd .. # should be out of test directory  
module load miniconda3/base
```

```
conda create --name day1  
conda activate day1
```

```
conda install -c conda-forge python numpy matplotlib -y
```

```
#Once install is complete  
conda deactivate
```


Running a job using the new environment

Submit the job

```
sbatch spiral.batch
```

Monitoring job status

```
squeue -u myBamaID
```

View the output

```
cat output_primes.%A
```

BACK IN SFTP WINDOW

Retrieve the .png file

```
get /home/myBamaID/test/spiral.png C:\Users\myBamaID\Desktop
```

```
#!/bin/bash
#SBATCH --job-name=spiraljob
#SBATCH --output=output_primes.%A
#SBATCH --error=error_primes.%A
#SBATCH --partition=main
#SBATCH --qos=main
#SBATCH --nodes=1
#SBATCH --mem=2G
#SBATCH --time=00:10:00
```

```
module purge
module load miniconda3/base ← Loading the module
```

```
conda activate day1 ← Activating the conda
                      environment
```

```
python ./primesspiral.py
```

Final checks

Check on your account

```
[myBamaID@uahpc-login001 ~]$ sacct -u myBamaID
```

View info on nodes and partitions

```
[myBamaID@uahpc-login001 ~]$ sinfo
```

Return a report about top users

```
[myBamaID@uahpc-login001 ~]$ sreport
```

sreport: user TopUsage

Exit the ssh session

```
[myBamaID@uahpc-login001 ~]$ exit
```

sreport: user TopUsage					

Top 10 Users 2026-06-24T00:00:00 - 2026-06-24T23:59:59 (86400 secs)					
Usage reported in CPU Minutes					

Cluster	Login	Proper Name	Account	Used	Energy
-----	-----	-----	-----	-----	-----
uahpc	ajmelles+	Aidan J. Melle+	jpanickacheril+	967936	0
uahpc	jzhang163	Jun Zhang	mkaminski3_grp	316338	0
uahpc	akraus	Alex Kraus	hturner_grp	154080	0
uahpc	knhorton1	Katie N. Horton	mrckain_grp	126720	0
uahpc	sislam23	S M Mohaimenul+	mpapon_grp	74072	0
uahpc	adejode	Aurelien De Jo+	bmtitus_grp	65833	0
uahpc	jepoulos	Jack E. Poulos	gjs_grp	61043	0
uahpc	aimran1	Ashik Imran	ahauser_grp	46080	0
uahpc	frezaiea+	Fatemeh Rezaei+	honoradkhani_grp	44640	0
uahpc	jppatton	Parker Patton	sdibbo_grp	38760	0

LUNCH BREAK

Please return by 1:30pm

Qualtrics survey

